



Rainfall

A binary exploitation project

42 Staff pedago@42.fr

Summary: This project is an introduction to the exploitation of ELF binaries.

Version: 5.0

Contents

I	Foreword	2
II	Introduction	3
III	Objectives	4
IV	General instructions	5
V	How progression works	7
VI	Mandatory part	9
VI.1	Boss fight: level10	10
VII	Bonus part	11
VIII	Submission and peer-evaluation	12

Chapter I

Foreword



There is something wrong...

Chapter II

Introduction

As a developer, you will write programs that run with privileges other people depend on. They parse input you do not control, sit on the stack and the heap you reason about only loosely, and trust assumptions the compiler never promised to keep.

A single unchecked copy, one integer that wraps, a format string fed straight to `printf`, and an attacker is no longer reading your program: they are **driving** it. Learning how a binary is bent into doing the attacker's bidding is the only reliable way to recognise these flaws in your own code and stop writing them.

Rainfall drops you onto a deliberately vulnerable machine and asks you to break it, one ELF binary at a time. You begin as `level100` and climb one account at a time. Each level is a small SUID program that mishandles its input; hidden behind it is a flag that only the next account is allowed to read, and that flag is the password that unlocks the following stage. To progress, you must read the binary, understand exactly how it lays out memory, and turn that understanding into control of execution.

Be warned: this project is demanding from the very first level. These are not puzzles with a trick to guess. They reward patient disassembly, careful reading of the stack, and a stubborn refusal to move on until you truly understand what the program does with every byte you feed it. Each level also tightens the screws. Protections you ignored early on (a non-executable stack, stack canaries, ASLR, full RELRO and PIE) are switched on one by one, and your exploits must defeat them.

There are no hints inside the machine beyond the binaries themselves. Read the code, use your tools, and let what you observe decide what you try next.



This project is deliberately complex. Getting stuck is part of the experience, not a sign of failure. Take your time and do not give up.

Chapter III

Objectives

Eleven levels. One VM. Each level teaches a different facet of binary exploitation on a real x86-64 program rather than a textbook exercise: stack buffer overflows, hidden functions, off-by-one errors, return-oriented programming and ret2libc, format-string leaks and writes, integer overflows, stack pivots, and, in the bonus, heap corruption and modern mitigation bypasses.

By the end you will be comfortable reading unfamiliar binaries, reasoning about how they handle their input and their privileges, and spotting the gap between what a program was meant to do and what it can actually be made to do. You will rely on the inspection and debugging tools already present on the machine (`gdb`, `objdump`, `readelf`, `ROPgadget`, `checksec`, ...). Working out which tool fits each situation is part of the challenge.

You may reach dead ends. Step back, re-read the disassembly, and look at the problem from a different angle. The answer is always in the binary: you just have to find it.



This project is done in a group. Different levels naturally suit different skill sets. Share what you find and work together.

Chapter IV

General instructions

- This project will be evaluated by humans.
- You may need to demonstrate your results during your evaluation. Be ready to walk through every step.
- To complete this project, import the provided .ova appliance into VirtualBox and start it. Once the VM has finished booting, its console will look like this:

```
Ubuntu 24.04.4 LTS rainfall tty1
rainfall login: level100
Password:
Welcome to Ubuntu 24.04.4 LTS (GNU/Linux 6.8.0-124-generic x86_64)

System information as of Tue Jun  9 10:33:26 PM UTC 2026

System load:  0.16           Processes:      131
Usage of /:   55.9% of 9.74GB Users logged in:   0
Memory usage: 9%            IPv4 address for enp0s3: 10.0.2.15
Swap usage:   0%

Rainfall

42 Network Security Challenge - Rainfall v2  @wil
level100@rainfall:~$ _
```

- Connect via SSH on port **4242** with the entry credentials: username level100 and password level100.

```
$> ssh level100@<vm-ip> -p 4242
level100@<vm-ip>'s password: level100
level100@rainfall:~$
```

- From your host machine, the server is reachable on the network. Identifying its exact address is straightforward with VirtualBox NAT.



You cannot modify the .ova file or create an altered copy under any circumstances. You must exploit the **running server**.

How progression works

- ```
level00@rainfall:~$ <exploit ./case>
$ cat /home/flag00/.pass
<flag string>
$ exit
level00@rainfall:~$ exit
```

```
$> ssh level01@<vm-ip> -p 4242
Password: <flag string>
level01@rainfall:~$
```

```
$> ssh level00@10.0.2.15 -p 4242
level00@10.0.2.15's password:
```

$$\overline{D} \setminus \{0\} \rightarrow \overline{D} \setminus \{0\}$$

42 Network Security Challenge – Rainfall v2 @wil

```
level00@rainfall:~$ ls
case case.c README
level00@rainfall:~$ cat README
The sky above the port was the color of television, tuned to a dead channel.
level00@rainfall:~$ file case
case: setuid ELF 64-bit LSB executable, x86-64, dynamically linked, not stripped
level00@rainfall:~$ id
uid=1001(level00) gid=1003(level00) groups=1003(level00),1001(levelgroup)
level00@rainfall:~$
```

- You cannot brute-force the SSH login. You must find the flag through the intended exploit path and be able to explain every step.



Brute forcing SSH is strictly forbidden and will result in an immediate failure of the evaluation.

# Chapter VI

## Mandatory part

- For the mandatory part, you must complete **level00 through level10** (11 levels, including the boss fight).
- Your repository must include everything needed to prove and explain your solution for each level. Use one folder per level:

```
$> ls
level00 level01 level02 level03 level04 level05
level06 level07 level08 level09 level10

$> ls level00/
flag resources/

$> cat level00/flag | cat -e
XXXXXXXXXXXXXXXXXXXXXXXXXXXX$
```

- The **flag** file contains the flag you recovered. The **resources/** folder contains any scripts or files you used to solve the level. The sources are provided on the VM, so you are not asked to reproduce them, only to show how you exploited them.
- You must be able to **clearly explain every step** of your solution during the evaluation.



Your repository must not contain any binary files. If you need a file from the VM, download it during the evaluation.

- If you use external tools, set up a proper environment (VM, Docker, Vagrant).
- Scripts that automate your solution are welcome, but you must be able to explain them in detail.

## VI.1 Boss fight: level10

Once you have completed `level09`, the flag you recover becomes the SSH password for `level10`. This is the final challenge of the mandatory path.

`level10` runs with every mitigation enabled at once (a non-executable stack, stack canaries and ASLR), and hands you no single primitive that breaks them all. You will have to chain several together: leak what you need at runtime, defeat the canary, rebuild a libc-relative ROP chain, and only then take control. The difficulty is higher than every previous level.

Include a `level10/` folder in your repository, with the same structure as the other mandatory levels (`flag + resources/`).



`level10` is part of the mandatory path. It will be assessed like any other mandatory level.

# Chapter VII

## Bonus part



Bonus will only be assessed if the mandatory part is **PERFECT**, including level10. If any level is incomplete or unexplained, bonus levels will not be evaluated.

Beyond the eleven mandatory levels, the server hosts **5 additional challenges** (bonus01 through bonus05). Most are hardened with the **modern mitigations** you met in the main chain and then some (NX, stack canaries, ASLR, full RELRO and PIE), while a couple relax one protection precisely because the technique they teach depends on it. Each uses a distinct approach that appears nowhere in the mandatory path: heap corruption, use-after-free, defeating PIE, dynamic-resolver abuse, and more. Read each binary carefully, because the protections it carries are part of the puzzle.

Be warned: the bonus levels are **significantly harder** than the mandatory ones. They assume you are already fluent with everything the main chain taught you, then push well past it. Do not expect them to fall quickly, and do not start them until your mandatory part is solid.

For each bonus level you complete, include a **bonusNN/** folder in your repository with the same structure as the mandatory levels: a **flag** file and a **resources/** folder.



The bonus chain starts where the mandatory one ends: the password of the first bonus user is the last mandatory flag. From there, you are on your own. Good luck :)

# Chapter VIII

## Submission and peer-evaluation

Submit your assignment to your `Git` repository as usual. Only the work inside your repository will be evaluated during the defense.

How you organize your **resources**/ folders is entirely up to you, as long as your work is easy to navigate and you can explain all of it during the evaluation. Double-check the names of your folders and files to ensure they are correct.